

Stellenbosch
UNIVERSITY
IYUNIVESITHI
UNIVERSITEIT

Backseat driver: Privileged-information auxiliary loss in autonomous racing

by

Jacques Friedrich Amsel

Report submitted in partial fulfilment of the requirements of the module Project (E) 448 for the degree Baccalaureus in Engineering in the Department of Electrical and Electronic Engineering at the University of Stellenbosch

Study leader: Prof. JC Schoeman

October 2026

Acknowledgements

Plagarism declaration

1. Plagiaat is die oorneem en gebruik van die idees, materiaal en ander intellektueleeiendom van ander persone asof dit jou eie werk is. *Plagiarism is the use of ideas, material and other intellectual property of another's work and to present is as my own.*
2. Ek erken dat die pleeg van plagiaat 'n strafbare oortreding is aangesien dit 'n vorm van diefstal is. *I agree that plagiarism is a punishable offence because it constitutes theft.*
3. Dienooreenkomstig is alle aanhalings en bydraes vanuit enige bron (ingesluit die internet) volledig verwys (erken). Ek erken dat die woordelikse aanhaal van teks sonder aanhalingstekens (selfs al word die bron volledig erken) plagiaat is. *Accordingly all quotations and contributions from any source whatsoever (including the internet) have been cited fully. I understand that the reproduction of text without quotation marks (even when the source is cited) is plagiarism.*
4. Ek verklaar dat die werk in hierdie skryfstuk vervat, behalwe waar anders aangedui, my eie oorspronklike werk is en dat ek dit nie vantevore in die geheel of gedeeltelik ingehandig het vir bepunting in hierdie module/werkstuk of 'n ander module/werkstuk nie. *I declare that the work contained in this assignment, except where otherwise stated, is my original work and that I have not previously (in its entirety or in part) submitted it for grading in this module/assignment or another module/assignment.*

26938839 Student number	 Signature
J.F. Amsel Initials and surname	18 October 2026 Date

Abstract

Table of contents

Acknowledgements i

Plagarism declaration ii

Abstract iii

Notation vii

Acronyms and abbreviations ix

1. Introduction 1

1.1. Background and motivation 1

1.2. Problem statement 2

1.3. Objectives 2

1.4. Summary of work 3

1.5. Scope 3

1.6. Roadmap 4

2. Background and Related Literature 5

2.1. Partially Observable Markov Decision Processes (POMDPs) 5

2.1.1. Value functions 6

2.2. Reinforcement Learning (RL) 7

2.3. Policy gradient methods 8

2.3.1. REINFORCE 10

2.3.2. Proximal Policy Optimization (PPO) 10

2.4. Auxiliary Learning in Policy-Based Methods 11

2.4.1. Auxiliary Losses 11

2.4.2. Auxiliary Rewards/Shaped Rewards 12

2.5. Loss scaling 13

2.5.1. Hyperparameter tuning 13

2.5.2. Uncertainty weighting 14

2.6. Huber loss 15

2.7. Linear Probing of Representations 15

2.8. Related work 16

3. System overview **18**

3.1. Simulator 19

3.1.1. Simulator performance comparison 19

3.1.2. Unity Vehicle Model 20

3.1.2.1. The Rigidbody 20

3.1.2.2. The WheelCollider 21

3.1.2.3. LiDAR 23

3.1.3. Track generation 23

3.1.3.1. Centerline generation 23

3.1.3.2. Track walls 25

3.2. Environment interface 26

3.3. The Agent 27

3.3.1. Action Distributions 28

3.4. Auxiliary Targets 29

3.5. Training 29

3.6. Metrics 30

3.6.1. Progress 30

3.6.2. Crash rates 30

3.6.3. Reward function 30

3.6.4. Area Under the Curve (AUC) 31

3.7. Trainer Performance Tuning 31

4. Experiments & results **33**

4.1. Probing intermediate layers for auxiliary information 33

4.2. Meeting questions 35

4.3. Learned features after auxiliary learning 36

4.4. Primary Task Performance 37

4.4.1. Variation in Primary Task Performancee 37

5. What I did **38**

6. Conclusions and recommendations **39**

Bibliography **40**

Appendix **46**

Appendix A WheelCollider properties 46
Appendix B Auxiliary Head Regression Results 47
Appendix C Training Performance Benchmark Results 48

Notation

γ	Discount rate
λ	GAE discount rate
r_t	Reward received at time t
$G_t = \sum_{i=t}^{T} \gamma^{i-t} \cdot r_i$	Discounted return at time t
$\mathcal{S}$	Set of all states
S_t	Random variable for state at time t
s_t	State at time t
$\mathcal{O}$	Observation space
O_t	Random variable for agent's observation at time t
o_t	Agent's observation at time t
$\mathcal{A}$	Action space
A_t	Random variable for action taken at time t
a_t	Observed action taken at time t
π_θ	Policy given parameters θ
$\pi_\theta(a_t \mid o_t)$	Probability density for action a_t given observation o_t and policy parameters θ
$\Delta_{\mathcal{Y}}(X)$	Set of all probability distributions over variable X with support $\mathcal{Y}$
$V^\pi(s) = \mathbb{E}_\pi[R_t \mid S_t = s]$	Value function for policy π at state s
$Q^\pi(s, a) = \mathbb{E}_\pi[R_t \mid S_t = s, A_t = a]$	Action-value function for policy π at state s and action a
$\hat{X}$	Estimator for random variable X

$\hat{x}$	Sample estimate of random variable X
$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$	Advantage function for policy π at state s and action a
$\mathrm{Var}(X)$	Variance of random variable X
$\rho_0(s)$	Initial state distribution
$X \sim Y$	Random variable X is distributed according to distribution Y
$X \dot{\sim} Y$	Random variable X is approximately distributed according to distribution Y
$\mathbb{R}$	The set of real numbers
$\mathbb{E}[X]$	Expected value of random variable X
$\mathbb{E}_{x\sim p}[f(x)]$	Expected value of function $f(x)$ when x is distributed according to p
$N(y;\mu;\sigma^2)$	Gaussian probability density function with mean μ and variance σ^2 with evaluation point y

Acronyms and abbreviations

LiDAR	Light Detection and Ranging
MDP	Markov Decision Process
MAE	Mean Absolute Error
MSE	Mean Squared Error
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimization
RL	Reinforcement Learning
RMSE	Root Mean Squared Error

Chapter 1

Introduction

1.1. Background and motivation

Road traffic crashes remain a leading cause of death and serious injury worldwide, and the overwhelming majority are attributed to human error [1], as cited in [2]. Autonomous driving promises to reduce severe and fatal accidents (e.g. severe lane-departure accidents [3]), improve traffic flow [4], and reduce emissions [5]. Before such systems can be trusted on public roads, however, they must handle difficult situations, where small mistakes compound quickly.

Autonomous racing is a demanding testbed for exactly these situations. The high speeds, tight control margins, and short time available to recover from bad decisions make it a proxy for developing and evaluating autonomous driving algorithms.

Generally, there exist two broad approaches to control in autonomous driving: end-to-end learning, where a model is trained to map sensor inputs (e.g. LiDAR, camera frames) directly to control outputs (e.g. speed and steering angle), and modular pipelines, where the problem is broken down into subproblems such as perception, planning, and control. End-to-end learning has the advantage of being able to learn a policy directly from data, without requiring hand-engineered features or intermediate representations. However, it can be difficult to interpret and debug, and may require large amounts of data to learn effectively. Modular pipelines, on the other hand, can be more interpretable and easier to debug, but may require more domain knowledge and engineering effort to design and tune [6].

Auxiliary learning is a technique in which a model is trained to perform additional (auxiliary) tasks alongside the primary task, in the hope that these tasks help it learn intermediate representations that are useful for the primary task. In reinforcement learning, auxiliary tasks can improve performance by providing additional supervisory signals during training [7], while also aiding the interpretability of the learned repre-

sentations [6]. This is attractive for end-to-end racing: a simulator knows the car's true velocity, slip and the geometry of the track ahead, so these quantities can supervise the network during training even though they will not be available to the deployed policy.

1.2. Problem statement

An end-to-end reinforcement learning (RL) agent that drives from LiDAR alone has to learn representations of privileged information that it cannot directly observe in the real world: its own velocity and slip, the curvature of the track ahead, and, in H2H racing, the opponent's location. Early in training, the agent has not yet learned to drive well enough to explore the track effectively, so the reward signal that would teach it these representations is sparse and noisy. This can lead to slow convergence.

Privileged information is, however, available in simulation. This report investigates whether *auxiliary losses* that predict this privileged information from the policy's shared representation can improve the learning of a LiDAR-only end-to-end racing agent. The research question is:

Does training with privileged-information auxiliary losses improve the performance or training speed of an end-to-end RL racing agent that drives from LiDAR alone, and does it change what the agent's representation encodes?

1.3. Objectives

The objectives of this Skripsie are to:

(a) select an appropriate simulation environment for autonomous racing;
(b) implement a baseline end-to-end RL agent that drives from LiDAR alone;
(c) gain an understanding of what representations the baseline agent natively encodes;
(d) implement auxiliary learning to provide additional supervisory signals to the agent during training; and
(e) evaluate the performance of the auxiliary learning agent against the baseline agent.

Meeting these objectives answers the research question: (a) and (b) provide the experimental platform and the baseline, (c) establishes what auxiliary learning must improve on, and (d) and (e) test the proposed method directly.

1.4. Summary of work

A custom racing simulator was built in Unity, using PhysX-based vehicle dynamics for the RoboRacer vehicle and a procedural track generator that produces a new closed track for each training episode. It was benchmarked against the F1Tenth gym and found to run only about 20% slower despite its higher-fidelity physics, and so it was chosen over the F1Tenth gym.

The baseline agent (a feedforward network) that takes a stack of 50 LiDAR scans, current steering and throttle as input was trained with PPO and a dense progress-based reward. To understand what this baseline encodes, linear regression probes were fitted to the activations of its final shared layer to predict privileged quantities. Even at random initialisation, some quantities are already linearly decodable: the car's lateral and angular velocity are predicted with mean out-of-sample R^2 of 92% and 85% respectively.

Auxiliary heads predicting forward, lateral and angular velocity, per-wheel slip, and upcoming track curvature were then added to the network and trained with a weighted auxiliary loss. When auxiliary heads for close- or medium-distance-ahead curvature ($<$ 3m and 3-6m ahead respectively) were introduced, the model encoded measurably more information on these quantities (up to $+0.5$ R^2 on average). Close track curvature heads resulted in improved training speed (+15.2% reward AUC on average), better performance after 600 PPO iterations (+12.4% average reward, +12.5% average velocity, and +12.26% average progress around the track per timestep in the final 50 iterations), and a collision rate per timestep 44% lower than the baseline. On the other hand, far-ahead (6-8m ahead) curvature heads resulted in worse performance than the baseline. We hypothesise that this is because far-ahead curvature is often not visible in the LiDAR scans, so the network is forced to learn an impossible task and, since the gradient norm is limited, the auxiliary loss dominates the primary loss, preventing useful policy updates.

Some wheel slip heads resulted in moderate performance improvements (e.g. -51.9% collision rate per timestep), but minor improvements in racing speed.

1.5. Scope

This report is limited to simulation; no sim-to-real transfer to a physical RoboRacer was attempted. The agent drives from LiDAR and its previous action only, and uses a

memoryless feedforward policy with a stacked observation history rather than a recurrent architecture. Learning is done with PPO alone, and other RL algorithms were not compared. Only auxiliary *losses* are studied; auxiliary rewards and reward shaping ([8]) are discussed as background but not evaluated. Auxiliary loss weights were chosen by fixed weighting rather than learned weighting, so methods such as uncertainty weighting [9] are discussed in the literature review but [not implemented / implemented: must update to match what was done before I hand in]. Representations are analysed with linear probes only, so non-linearly encoded information is not measured. H2H racing is not studied, so opponent information is not considered.

1.6. Roadmap

Chapter 2 reviews the background and related literature: POMDPs and value functions, policy gradient methods up to PPO, auxiliary losses and rewards, loss weighting, and linear probing of representations, and closes with related work on auxiliary tasks in RL.

Chapter 3 describes the system: the Unity simulator and its comparison with the F1Tenth gym, the vehicle and tyre model, procedural track generation, performance tuning of the trainer, and the reward function and evaluation metrics, as well as the auxiliary targets and the network architecture.

Chapter 4 presents the experiments and results: linear probing of the baseline, the change in probed representations after auxiliary learning, and the effect on primary task performance.

Chapter 5 concludes by showing how the objectives were met and makes recommendations for future work.

Chapter 2

Background and Related Literature

2.1. Partially Observable Markov Decision Processes (POMDPs)

Many control and robotics problems require an agent to make a sequence of decisions in an environment, where each decision influences what happens next and where the agent can sense only part of the environment. We model such problems as a *partially observable Markov decision process* (POMDP) defined by the tuple $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{P}, \rho_0, \mathcal{R}, \gamma, \mathcal{Z})$ [10], whose interaction loop is shown in Figure 2.1.

At each time step t, the environment is in state $s_t \in \mathcal{S}$, where $\mathcal{S}$ is the state space. This state is a *complete* description of the environment at that time. The first state is sampled from an initial state distribution, $s_0 \sim \rho_0$. The agent does not see s_t directly. Instead, it receives an observation $o_t \in \mathcal{O}$, typically its sensor inputs (e.g. LiDAR, camera frames), drawn from the observation function, $o_t \sim \mathcal{Z}(\cdot \mid s_t)$, where $\mathcal{O}$ is the observation space. The information in s_t that is missing from o_t is called *privileged information*. For example, a car may observe its lidar measurements, but not the road surface conditions directly.

Based on its observation, the agent uses its policy π to choose an action $a_t \in \mathcal{A}$, where $\mathcal{A}$ is the action space. The policy maps an observation to a probability distribution over actions; we write $a_t \sim \pi(\cdot \mid o_t)$. The policy is what we want to find: it describes the agent's behaviour, whereas the POMDP describes the environment in which the agent acts. For simplicity, we let the policy depend only on the current observation; should the policy depend on a history of observations, we would write $\pi(\cdot \mid o_t, o_{t-1}, ...)$. After the agent has taken action a_t, the environment moves to a new state, $s_{t+1} \sim P(\cdot \mid s_t, a_t)$, where P is the transition function. By the *Markov property*, the next state depends only

on the current state and action, not on earlier history [11]. Then, the agent receives a scalar reward $r_t = R(s_t, a_t, s_{t+1})$, where R is the reward function, together with the next observation o_{t+1}. The agent's goal is to maximise the expected discounted return. The *return* from time step t is the discounted sum of all rewards the agent receives from t onward,

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k},$$

where T is the final time step and $\gamma \in [0, 1)$ is the discount factor. A reward received k steps ahead is scaled by γ^k, so later rewards are worth less than immediate ones: when γ is close to 1 the agent is far-sighted, and when γ is close to 0 it is short-sighted. Since G_t depends on random states, observations and actions, it is a random variable, and the agent maximises its expectation $\mathbb{E}[G_0]$ from the start of the episode. The policy that achieves this is the *optimal policy* π^*.

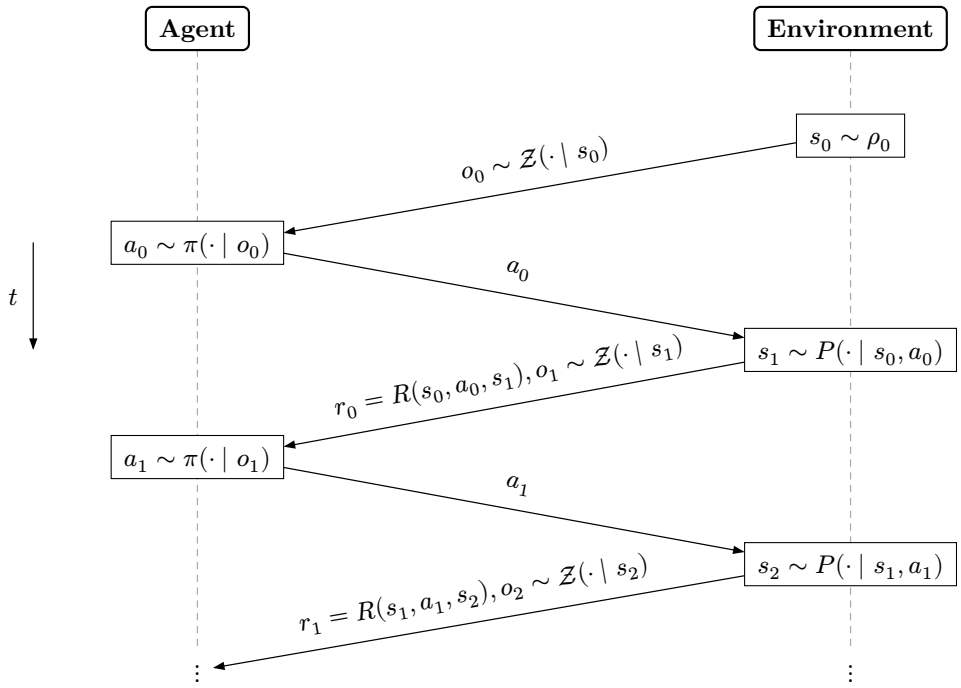


Figure 2.1: Interaction between the agent and the environment over time (t).

2.1.1. Value functions

Throughout this section we assume a memoryless policy $\pi(\cdot \mid o_t)$. Since $o_t = \mathcal{Z}(s_t)$, the agent's action distribution in state s is $\pi(\cdot \mid \mathcal{Z}(s))$, and because the state is Markov, the expected return from time t depends only on s_t. The value functions below are therefore well defined as functions of the state.

Value functions are a central concept in reinforcement learning, providing a measure of how good it is for the agent to be in a particular state or to take a particular action. The state-value function,

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

represents the expected discounted return, the agent can achieve starting from state s and following policy π thereafter. The action-value function,

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

represents the expected discounted return the agent can achieve starting from state s, taking action a, and following policy π thereafter. The advantage function,

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s) \tag{1}$$

measures the relative value of taking action a in state s compared to the average value of all actions in that state.

These state-value and action-value functions satisfy the Bellman equations, which express the relationship between the value of a state and the values of its successor states. [12] The Bellman equation for the state-value function is given by:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V^{\pi}(S_{t+1}) \mid S_t = s] \tag{2}$$

and the Bellman equation for the action-value function is given by:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[R_{t+1} + \gamma Q^{\pi}(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a] \tag{3}$$

These Bellman equations Equation 2 and Equation 3 provide a recursive relationship that allows the agent to update its value estimates based on the observed rewards and the estimated values of successor states.

2.2. Reinforcement Learning (RL)

Reinforcement learning (RL) is a subfield of machine learning concerned with finding a policy π that maximises the expected discounted return $J(\pi) = \mathbb{E}_{\pi}[G_0]$ using only data sampled from interaction with the environment, without labelled optimal actions or a known model. RL differs from supervised learning in that feedback is evaluative rather than instructive: the agent is told how good its actions were, not which action was correct, and rewards may arrive long after the actions that caused them. Furthermore, the data distribution depends on the agent's own policy, so the agent must explore.

RL algorithms fall into three broad classes. *Value-based* methods learn the optimal action-value function $Q^*(s,a) = \max_\pi Q^\pi(s,a)$, which satisfies the Bellman optimality equation

$$Q^*(s,a) = \mathbb{E}\Big[r_t + \gamma \max_{a' \in \mathcal{A}} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a\Big],$$

and act greedily with respect to it, $\pi(s) = \arg\max_{a \in \mathcal{A}} Q^*(s,a)$. In a fully observable MDP this greedy policy is optimal, but the agent is restricted to deterministic policies (which are not always optimal in partially observed problems), and the maximisation over $\mathcal{A}$ can be intractable for continuous actions [13]. In a POMDP, the observation is not necessairily Markov, so $Q(o,a)$ does not satisfy a Bellman equation, and a state-based $Q(s,a)$ cannot define a deployable policy as the agent does not observe s. *Policy-based* methods, on the other hand, learn a parameterised policy π_θ directly by ascending the gradient of $J(\pi_\theta)$ with respect to θ, and do not need the Markov property of the observations or an $\arg\max$ over actions. They can also represent stochastic policies, which may be necessary for optimal behaviour in some POMDPs [14] (although there always exists a deterministic optimal policy in fully-observed MDPs). *Actor-critic* methods combine both: a learned value function (the critic) is used to reduce the variance of the policy gradient, and since the critic is only used in training, it may condition on privileged information. We focus on policy-based and actor-critic methods in this report [11].

2.3. Policy gradient methods

Policy gradient methods are a class of RL training algorithms that learn a policy π_θ with parameters θ directly by maximising the expected return $\mathbb{E}[G_0]$ with respect to the policy parameters θ. Given a trajectory $\tau = (s_0, o_0, a_0, r_0, s_1, o_1, a_1, r_1, ..., s_{T-1}, o_{T-1}, a_{T-1}, r_{T-1}) \sim p_\theta$, where p_θ is the probability of the trajectory when following the policy π_θ, we intend to maximise the expected return with respect to the policy parameters θ:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[G_0] = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right] = \int p_\theta(\tau) G(\tau)\, \mathrm{d}\tau$$

[1]This can be done only under certain regularity conditions not explored in this report, but generally if the functions are continuous and differentiable everywhere, and the integral is finite.

Using the *log-derivative* trick, $\nabla_\theta p_\theta(\tau) = p_\theta(\tau)\nabla_\theta \log p_\theta(\tau)$, and assuming that the integral and derivative can be interchanged[1], the gradient of the expected return with respect to θ is then

$$\nabla_\theta J(\theta) = \nabla_\theta \int p_\theta(\tau) G(\tau)\, \mathrm{d}\tau = \int \nabla_\theta p_\theta(\tau) G(\tau)\, \mathrm{d}\tau = \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) G(\tau)\, \mathrm{d}\tau$$

$$= \mathbb{E}_{\tau \sim p_\theta}[\nabla_\theta \log p_\theta(\tau) G(\tau)]$$

Considering that the probability of a trajectory can be factorised as

$$p_\theta(\tau) = p(s_0) \prod_{t=0}^{T-1} \underbrace{p(o_t \mid s_t)}_{\mathcal{Z}} \cdot \pi_\theta(a_t \mid o_t) \cdot \underbrace{p(r_t \mid s_t, a_t, s_{t+1}, o_t)}_{\mathcal{R}} \cdot \underbrace{p(s_{t+1} \mid s_t, a_t)}_{\mathcal{P}}$$

since $p(s_0)$, $\mathcal{P}$, $\mathcal{Z}$ and $\mathcal{R}$ do not depend on θ, their gradients vanish, so

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid o_t)$$

Then, the *policy gradient* can be expressed as

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid o_t) G(\tau)\right]$$

Write $G(\tau) = G_0$. Splitting the return at time t,

$$G_0 = \sum_{k=0}^{t-1} \gamma^k r_k + \gamma^t G_t, \quad G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k,$$

where G_t is the discounted *reward-to-go*,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid o_t) \left(\sum_{k=0}^{t-1} \gamma^k r_k + \gamma^t G_t\right)\right]$$

However, since the expectation of the fixed past-reward terms is zero, they can be removed from the policy gradient without introducing bias. This can be seen by conditioning on the history $h_t = (s_0, o_0, a_0, r_0, ..., s_{t-1}, o_{t-1}, a_{t-1}, r_{t-1}, s_t, o_t)$:

$$\mathbb{E}_{a_t \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a_t \mid o_t) \mid h_t] = \int_a \nabla_\theta \pi_\theta(a \mid o_t) = \nabla_\theta \int_a \pi_\theta(a \mid o_t) = \nabla_\theta 1 = 0,$$

The past-reward terms therefore vanish in expectation, leaving

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T-1} \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid o_t) G_t\right].$$

Removing these zero-mean terms typically reduces the variance of the Monte Carlo estimate. In practice the factor γ^t is usually dropped; this yields a biased estimate of $\nabla_\theta J(\theta)$ for $\gamma < 1$ [15].

2.3.1. REINFORCE

The REINFORCE algorithm for policy learning was introduced by Williams in 1992 in the journal *Machine Learning* [16]. It is one of the simplest policy gradient methods: by sampling N trajectories $\tau^{(i)} \sim \pi_\theta$ of length T_i timesteps each, the policy gradient can be estimated in a Monte-Carlo fashion:

$$\widehat{\nabla_\theta J}(\theta) = \frac{1}{\sum_{i=0}^{N} T_i} \sum_{i=1}^{N} \sum_{t=0}^{T_i - 1} \log\Big(\pi_\theta\Big(a_t^{(i)} \mid o_t^{(i)}\Big)\Big) \cdot G_t^{(i)}$$

In order to run REINFORCE in an AutoGrad engine (such as PyTorch or TensorFlow), a loss function $\mathcal{L}_{\text{REINFORCE}}$ is defined such that its gradient with respect to the policy parameters θ is equal to the policy gradient estimate above. This loss function is given by:

$$\mathcal{L}_{\text{REINFORCE}} = -\frac{1}{\sum_{i=0}^{N} T_i} \sum_{i=1}^{N} \sum_{t=0}^{T_i - 1} \log\Big(\pi_\theta\Big(a_t^{(i)} \mid o_t^{(i)}\Big)\Big) \cdot G_t^{(i)}$$

Although REINFORCE is simple to implement, it has several drawbacks. First, it has high variance in its gradient estimates, which can lead to slow convergence and instability during training. Second, it does not make use of a value function to estimate the expected return, which can further increase variance. Finally, it is an on-policy algorithm, meaning that it can only learn from trajectories generated by the current policy, which can limit exploration and learning efficiency (since it cannot train multiple epochs on the same rollout).

2.3.2. Proximal Policy Optimization (PPO)

PPO is a modern algorithm for policy learning introduced in 2017 by researchers at OpenAI [17]. It improves on REINFORCE in two ways. First, it is an actor-critic method: alongside the policy, it learns a value function $V^\pi(s)$, which is used to compute an advantage estimate $\hat{A}_t$ and so reduces the variance of the policy gradient estimate. Second, it optimises a *clipped surrogate objective*, $\mathcal{L}_{\text{PPO}}$, that limits how far each update can move the policy. This prevents large, destructive steps and makes several epochs of updates on the same rollout possible.

Recall that the advantage function in Equation 1 is the difference between the action-value and state-value functions. It therefore measures how much better (positive) or worse (negative) an action is than the policy's average action in that state. In training, $\hat{A}_t(s_t, a_t)$ is estimated from observed rewards and the learned value function, $\hat{V}^\pi(s)$, for

example with Generalized Advantage Estimation [18]. In PPO, the clipped surrogate objective is given by

$$\mathcal{L}_{\text{PPO}} = -\frac{1}{\sum_{i=0}^{N} T_i} \sum_{i=1}^{N} \sum_{t=0}^{T_i - 1} \min\Big(r_t(\theta) \cdot \hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon) \cdot \hat{A}_t \Big)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{\theta_{\text{old}}}(a_t \mid o_t)}$ is the ratio of the new policy's probability of taking action a_t given observation o_t to the old policy's, and ε is a hyperparameter that controls how much the policy is allowed to change in a single update. The clip function ensures that a policy update that would make $r_t(\theta)$ fall outside of the interval $[1-\varepsilon, 1+\varepsilon]$ is clipped to the nearest endpoint of that interval. This prevents large policy updates that could destabilise training. The min function on the other hand ensures that, while the policy's update is capped when an update makes a good action ($A_t > 0$) more likely, it is not protected from penalties if an update makes a bad action ($A_t < 0$) more likely.

2.4. Auxiliary Learning in Policy-Based Methods

In RL, the *primary task* is to update a policy π_θ to maximises the expected discounted return $\mathbb{E}_{\tau \sim \pi_\theta}[G_0]$. This primary task is typically formulated as minimising a loss (e.g. $\mathcal{L}_{\text{PPO}}$) in the policy-based approach. *Auxiliary learning* augments this task with *auxiliary tasks* whose learning signals are used to improve the agent's representations. Unline general multi-task learning, perfromance on the auxiliary tasks is not itself the goal, and the auxiliary head outputs are typically discarded at inference time. Because auxiliary tasks can supply training signal when the reward is sparse or delayed [19], they can improve sample efficiency, and, by acting as an inductive bias[2], generalisation [21]. We consider two formulations: auxiliary **losses**, which leave the RL objective unchanged and shape only the shared representation, and auxiliary **rewards**, which modify the objective itself and can therefore change the optimal policy.

2.4.1. Auxiliary Losses

In this formulation, the agent is trained on a weighted sum of the primary loss and K auxiliary losses. Each loss is computed by a separate *auxiliary head* that reads from a representation shared with the policy. Writing the parameters as $\theta = \left(\theta_{\text{s}}, \theta_\pi, \theta_{\text{a}}^{(1)}, ..., \theta_{\text{a}}^{(K)}\right)$, where θ_{s} are the shared (encoder) parameters, θ_π the policy head and $\theta_{\text{a}}^{(k)}$ the k-th auxiliary head, the total loss is

[2]Inductive bias is the set of assumptions a learning algorithm makes to predict outputs given inputs that it has not encountered before. [20]

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{primary}}(\theta_{\text{s}}, \theta_{\pi}) + \sum_{k=1}^{K} w^{(k)} \mathcal{L}_{\text{aux}}^{(k)}\left(\theta_{\text{s}}, \theta_{\text{a}}^{(k)}\right),$$

where $w^{(k)} \geq 0$ weights each auxiliary task relative to the primary loss (only relative weights matter, so the primary weight is fixed to 1). The gradients with respect to each parameter group are

$$\nabla_{\theta_{\pi}} \mathcal{L}_{\text{total}} = \nabla_{\theta_{\pi}} \mathcal{L}_{\text{primary}},$$

$$\nabla_{\theta_{\text{a}}^{(k)}} \mathcal{L}_{\text{total}} = w^{(k)} \nabla_{\theta_{\text{a}}^{(k)}} \mathcal{L}_{\text{aux}}^{(k)}, \quad \nabla_{\theta_{\text{s}}} \mathcal{L}_{\text{total}} = \nabla_{\theta_{\text{s}}} \mathcal{L}_{\text{primary}} + \sum_{k=1}^{K} w^{(k)} \nabla_{\theta_{\text{s}}} \mathcal{L}_{\text{aux}}^{(k)}.$$

Only the shared parameters θ_{s} receive gradients from both the primary and auxiliary losses, which is how the auxiliary tasks influence the policy: they shape the representation it is built on

The primary loss, such as $\mathcal{L}_{\text{PPO}}$, is still a policy-gradient surrogate with a high-variance gradient estimate. Auxiliary losses, by contrast, are usually ordinary supervised or self-supervised objectives (for example predicting the next observation, or the immediate reward), so their gradients are exact and low-variance. This is what makes them useful when rewards are sparse or delayed. Their cost is that the auxiliary and primary gradients can conflict on the shared parameters, θ_{s} so the weights $w^{(k)}$ must be chosen with care.

2.4.2. Auxiliary Rewards/Shaped Rewards

Alternatively, the auxiliary task can be incorporated into the reward. The agent is given the modified reward

$$r_t^{\text{total}} = r_t^{\text{primary}} + \sum_{k=1}^{K} w^{(k)} r_t^{\text{aux},(k)},$$

where each $r_t^{\text{aux},(k)}$ rewards behaviour that is expected to help with the primary task, such as an intrinsic curiosity bonus [22] or progress towards a subgoal. The policy is then trained to maximise the expected return of r_t^{total} with the same policy gradient machinery as before.

Unlike auxiliary losses, this changes the objective being optimised: the agent now solves a different (PO)MDP, whose optimal policy need not be the same as π^* for the primary reward. Poorly chosen auxiliary rewards can therefore lead the agent to exploit them at the expense of the primary task (*reward hacking*). The exception is *potential-*

based shaping, where the auxiliary reward has the form $\gamma\Phi(s_{t+1}) - \Phi(s_t)$ for some potential function Φ, which is guaranteed to leave the optimal policy unchanged [8].

The auxiliary rewards do influence the policy, but only indirectly through the return G_t, and therefore through the same high-variance score-function estimator as the primary reward. No exact gradient is available, and the agent must work out through its own exploration which auxiliary rewards actually contribute to the primary task. This credit-assignment problem is harder still when the auxiliary rewards are themselves sparse or delayed.

2.5. Loss scaling

When composing the L_{total}, the question naturally arises: what should be the values of the weights $\{w^{(1)}, ..., w^{(N)}\}$? Since the values cannot simply be learnt by gradient descent (they would simply shrink to 0 or go negative to lower $\mathcal{L}_{\text{total}}$), we must use another technique.

2.5.1. Hyperparameter tuning

One may empirically estimate the optimal weights using hyperparameter tuning tools such as Optuna [23], or a simple grid search. However, when it is expensive to train a model, this method of search quickly becomes impractical, since every candidate set of weights requires its own training run.

Hyperparameter tuners differ in how they choose which weights to try next. Grid search evaluates every combination on a fixed N-dimensional grid, so its cost grows exponentially with the number of weights N. Random search samples configurations at random and is often more efficient than grid search, because usually only a few hyperparameters matter much [24]. Bayesian optimisation methods go further by building a model of how the score depends on the hyperparameters and using it to choose promising candidates. Examples include Gaussian-process-based optimisation [25] and the tree-structured Parzen estimator (TPE) [26], which is the default sampler in Optuna. Methods such as Hyperband [27] and its asynchronous variant ASHA [28] give each trial a small budget and only continue the best-performing ones. However, this method may kill RL trials that perform more exploration early on, so are not ideal for such scenarios.

Even with these savings, each trial still involves training a model, and the number of trials needed grows with the number of loss terms N. For multi-task models that

are expensive to train, this motivates methods that learn the weights during training instead.

2.5.2. Uncertainty weighting

[Will only include this if I have time to implement UW] Two kinds of uncertainty can be modelled in a model's predictions [29]:

- *Epistemic* uncertainty: the model is unsure because it hasn't been trained on enough data. More data reduces epistemic uncertainty.
- *Aleatoric* uncertainty: the data itself is noisy or incomplete, so even a perfect model can't predict exactly. In this case, more training data doesn't help.

Aleatoric uncertainty has two forms:

- *Data-dependent* or *Heteroscedastic*: the noise changes from input to input (e.g. a blurry image is harder to make inference from than a clear one).
- *Task-dependent* or *Homoscedastic*: the noise is the same for every input and depends only on the task

Kendall et al. propose a solution to determining the weights $\{w_1, ..., w_N\}$ problem by modelling the homoscedastic (task-dependent) uncertainty only [9].

Instead of formulating the prediction for task i, $\hat{y_i} = f_i(\boldsymbol{x})$, as a point estimate, it is modelled as a Gaussian with mean $f_i(x)$ and variance σ_i^2:

$$\hat{Y}_i \sim p(y_i \mid f_i(x)) = N(y_i; f_i(x); \sigma_i^2)$$

The negative log-likelihood of this Gaussian probability density function evaluated at the true value y_i is then the loss value for task i:

$$L_i = -\log p(y_i \mid f_i(x)) = \frac{1}{2\sigma_i^2} \|y_i - f_i(x)\|_2^2 + \log \sigma_i$$

A similar derivation can be made for L1 loss functions by assuming the predictions are $Y_i \sim \text{Laplace}(f_i(x), b_i)$:

$$L_i = -\log p(y_i \mid f_i(x)) = \frac{1}{b_i} \|y_i - f_i(x)\|_1 + \log b_i$$

Then, assuming the tasks are independent, the total loss is the sum of the individual losses. The log-variance $\log \sigma_i$ or log-scale parameter $\log b_i$ is a learnable parameter that is updated during training, allowing the model to learn the optimal weights for each loss term.

Of course, in reality, the tasks are not independent, but this method has empirically been shown to work well in practice [9]. With some caveats like the fact that it produces

a bias towards easy tasks, since they result in smaller σ_i values. This has led to the development of other methods not explored in this Skripsie, such as so-called Optimal Uncertainty Weighting [30].

2.6. Huber loss

The Huber loss [31] is a robust alternative to the squared-error (L2) loss that is far less sensitive to outliers. It behaves quadratically for small errors and linearly for large ones, with the transition point set by a threshold parameter β. As visualised in Figure 2.2, this means the gradient magnitude is bounded no matter how large the prediction error grows, whereas the L2 gradient grows without limit. Unlike the L1 loss, whose gradient changes sign abruptly at zero, the Huber loss is smooth there, so the gradient shrinks gradually as the error approaches zero. The Huber loss is defined as:

$$\mathcal{L}^{\beta}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 \text{ if } |y - \hat{y}| \leq \beta \\ \beta \, |y - \hat{y}| - \frac{1}{2}\beta^2 \text{ otherwise} \end{cases}$$

In actuality, PyTorch's implementation of the Huber loss (`smooth_l1_loss`) uses a rescaled definition, $\frac{\mathcal{L}^{\beta}(y,\hat{y})}{\beta}$, to prevent the β parameter from affecting the gradient in the linear region.

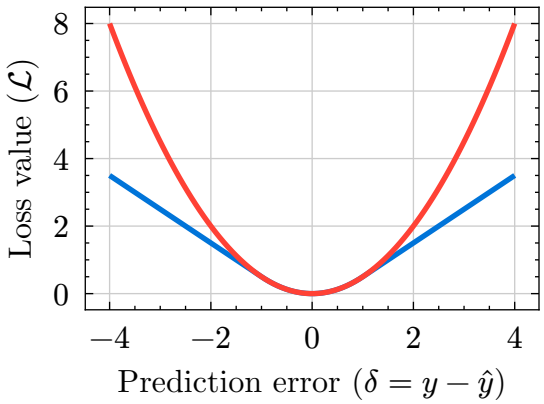


Figure 2.2: Huber loss (blue) vs squared error loss (red) for prediction errors δ.

2.7. Linear Probing of Representations

Consider the network in Figure 2.3. Once trained, each layer l maps an input $\boldsymbol{x}$ to an intermediate representation $\boldsymbol{h}^{(l)} = f_l(\boldsymbol{x}; \theta)$. We would like to know what information these representations encode, and in particular whether they encode privileged state variables (e.g. velocity, slip, upcoming curvature) that.

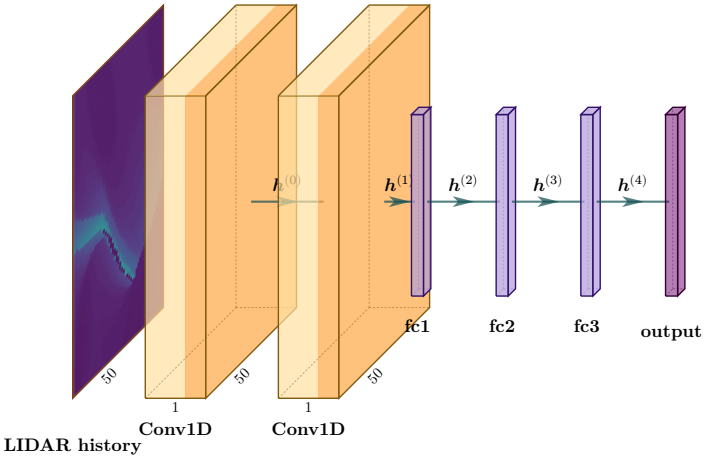


Figure 2.3: Example network architecture

Alain and Bengio [32] introduced *linear probes* for this purpose. The main network's weights are frozen, and a separate linear classifier (the probe) is trained on the representations $\boldsymbol{h}^{(l)}$ of each layer to predict the desired target. Since the probe cannot change the representation, its test accuracy measures how much class information is *linearly* available at that layer. Anand et al. [33] extended this idea to reinforcement learning, probing an encoder's representations for ground-truth state variables of Atari games (read from the console's RAM).

The choice of a *linear* probe is deliberate. A sufficiently expressive probe (e.g. a deep MLP) may compute the target from almost any input, including the raw network input, so high probe accuracy would say more about the probe than about the representation [34]. Restricting the probe to a linear map means that it can only succeed if the network has already done the work of encoding the target along some direction in the representation space.

Many of the privileged variables of interest are continuous rather than categorical, so we use a linear *regression* probe instead of a classifier, as done by McGrath et al. [35] to probe AlphaZero for chess concepts and by Gurnee and Tegmark [36] to probe language models for spatial and temporal encodings.

2.8. Related work

Lample and Chaplot in [37] trained Deep Q-Network (DQN) and Deep Recurrent Q-Network (DRQN) agents to play Doom from first-person game frames in 2017. They used auxiliary heads to predict binary labels for the presence of enemies in the field of view, which roughly doubled the kill-to-death ratio. However, the gain was only

significant for the recurrent DRQN, not for the memoryless DQN with stacked frames. The method was limited to discrete actions, vision only, and a single game.

Dosovitskiy and Koltun [38] proposed Direct Future Prediction (DFP) in 2017, which predicts measurements (health, ammo, frags) at several future offsets with single frames as input, using a supervised regression loss. The agent picks the action maximising a weighted sum of the predictions. This method outperformed Lample and Chaplot by more than 50% in k/d ratio, earning first place in the Visual Doom AI Competition (Full Deathmatch). They found that predicting multiple targets improved performance, even when those targets were not used in action selection. However, since it requires and argmax over the actions, it is intractable for continuous control. It is also not a RL method, as it receives no reward signal in training.

Mirowski et al. in [39] trained an recurrent LSTM A3C agent to navigate 3D mazes from RGB frames with sparse rewards and randomised goal locations. They used auxiliary heads for depth prediction and loop-closure classification[3]. They observed a 5x training speedup, and depth prediction from the last LSTM layer outperformed depth prediction at the output of the convolutional encoder block. Further, they found that an auxiliary loss for depth prediction was more effective than depth as an input.

Shelhamer et al. in [40] showed that representation learning is the bottleneck in RL and that self-supervised auxiliary losses can speed it up. They used A3C with auxiliary tasks built from the agent's own transitions: reward binning, inverse dynamics, and VAE or BiGAN[4]. They found that multi-task pre-training gave about 2.7x training efficiency (AUC) over the first 10M updates vs the baseline.

The previous work on auxiliary learning in RL has found it to be effective in improving sample efficiency and performance, especially in partially observable environments such as Doom in [37], [38] and with recurrent neural networks [37], [39]. However, most of the work has focused on visual inputs and discrete action spaces. We apply auxiliary learning to a continuous control problem of autonomous racing from LiDAR using a memoryless policy.

[Should I cite regular RL papers in F1Tenth that are not auxiliary learning? Feels useless...]

[3]Loop closure occurs when the agent returns to a previously visited location

[4]Variational autoencoders (VAEs) and bidirectional generative adversarial networks (BiGANs) are unsupervised generative models that learn a compact latent representation of their input without labels, and are trained so that the representation retains enough information to reproduce or model the data distribution.

Chapter 3

System overview

The system trains an end-to-end RL agent to drive a simulated RoboRacer vehicle from LiDAR. It runs entirely in simulation and has two subsystems, shown in Figure 3.1: a Unity simulator, which models the vehicle, track and physics interactions, and a Python training script, which computes rewards given the state communicated by the simulator, and updates the policy. The two communicate over gRPC through the ML-Agents toolkit [41], [42].

The subsystems advance in lockstep, following the interaction loop in Figure 2.1. At each timestep, the simulator sends each agent's observation, along with privileged state information, to the training script. The script replies with the action each vehicle should take next. The privileged information is used only during training and is never given to the policy, so the trained agent can drive from LiDAR alone. Rewards are computed in Python rather than in Unity, so the reward function can be changed without rebuilding the simulator.

To speed up data collection, the simulator is replicated: each Unity instance contains `num_areas` independent training areas - each with its own unique agent, and each trainer runs `num_inst` instances in parallel (the stacked boxes in Figure 3.1). The rest of this chapter describes the components in turn.

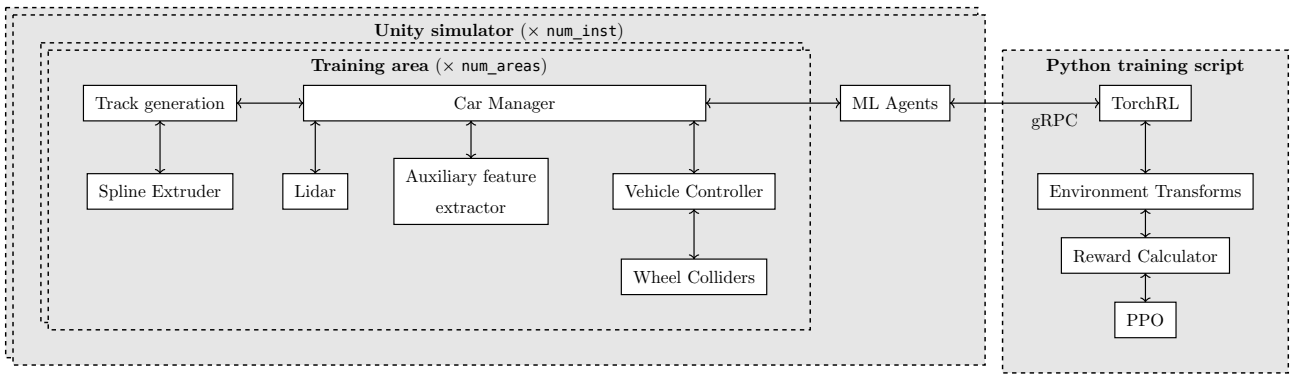


Figure 3.1: System diagram

3.1. Simulator

Previous work on end-to-end driving has largely made use of the F1Tenth gym, with the F1Tenth gym being the most common simulator among methods suveyed in [43]. The F1Tenth gym is a Python-based 2D simulator for the F1Tenth (now RoboRacer) vehicle. This simulator is easy to use, and runs in Numba-compiled Python, but features a simplified bicycle model of the vehicle, which approximates slip dynamics using a linear function [44].

The AutoDrive simulator [45], which uses the Unity game engine, is a 3D simulator that models the vehicle and its environment using PhysX, a physics engine developed by NVIDIA. PhysX simulates rigid-body dynamics, including collisions, friction and gravity, and allows for higher-fidelity simulations, including weight transfer dynamics, and suspsension deformation.

Table 3.1: Feature comparison of the Unity (PhysX) and F1Tenth gym simulators

	Unity (PhysX)	**F1Tenth gym**
Integrator	Runge-Kutta 4th order	Euler
Car model	3D, 4-wheel rigid body with suspension, weight transfer, non-linear wheel slip dynamics	2-wheel bicycle model with linearised slip dynamics
Sensors	LiDAR (max 101 rays per agent), IMU, wheel encoders[5], occupancy grids	LiDAR, IMU (excl. pitch, roll)
Track generation	No	In Beta [46]

However, the AutoDrive simulator by default does not provide various features that are important for end-to-end training in our auxiliary learning setup. For example, it is missing track generation & privileged information reporting. Therefore, a custom simulator was implemented in Unity using components from the AutoDrive simulator. Specifically, the `VehicleController` and `IMU` components were not re-implemented, but instead used directly from the AutoDrive simulator.

3.1.1. Simulator performance comparison

In order to ensure the custom Unity simulator is suitable for training and not just evaluation, its performance was compared to the F1Tenth gym. Both environments

[5]The IMU and wheel encoders are modules available in the AutoDrive simulator, not native Unity

were configured for 100Hz physics and 25Hz control with one vehicle per environment. Then, a rollout of 10 000 control timesteps (400 sim-seconds) was performed in each environment on an M4 MacBook Air, and the average time per control step was measured. Both environments were run in headless mode.

Despite the Unity simulator being significantly more complex than the F1Tenth gym, it runs at a comparable speed in this environment, as shown in Figure 3.2. The Unity simulator is slower than the F1Tenth gym by 20%. The memory usage of the Unity simulator is also higher than that of the F1Tenth gym, but this is a marginal diffence of 115 MB on average.

Due to the relatively minor performance difference, and Unity's more realistic 3D physics [Do I need to cite here/prove it? I feel as though it's obvious.], the Unity simulator was chosen for training the RL agent.

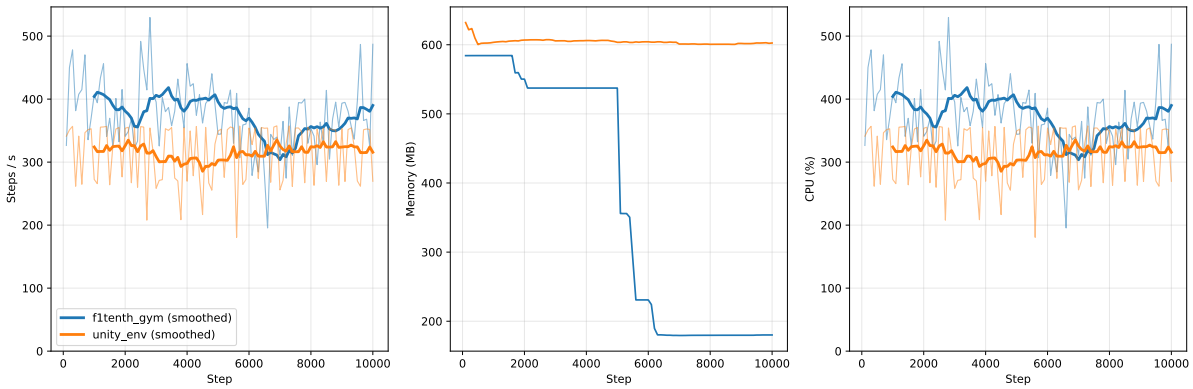


Figure 3.2: Benchmark of Unity simulator vs F1Tenth gym. Note that memory y-axis does not start at 0 to aid interpretability.

3.1.2. Unity Vehicle Model

The vehicle is modelled as a `Rigidbody` with four wheels, each simulated by a `WheelCollider`. The two components divide the work: the `Rigidbody` represents the car as a single moving mass, and the `WheelCollider`s generate the forces that move it. Near the front of the car, a `RayPerceptionSensor` from the ML-Agents toolkit simulates a 2D LiDAR sensor with 101 rays, which is used as the agent's observation.

3.1.2.1. The Rigidbody

A `Rigidbody` is the Unity component that gives an object physical behaviour. It stores the car's mass, centre of mass and moment of inertia. At each fixed timestep the physics engine (PhysX) integrates the forces and torques acting on it to update its linear and angular velocity and its pose. The car's body is covered by simple box colliders

(Figure 3.3), which handle collisions with walls and other objects. They play no part in the tyre-road interaction, which is handled entirely by the wheels. `BoxColliders` were used to model the car's body and collisions between the wheels and walls.

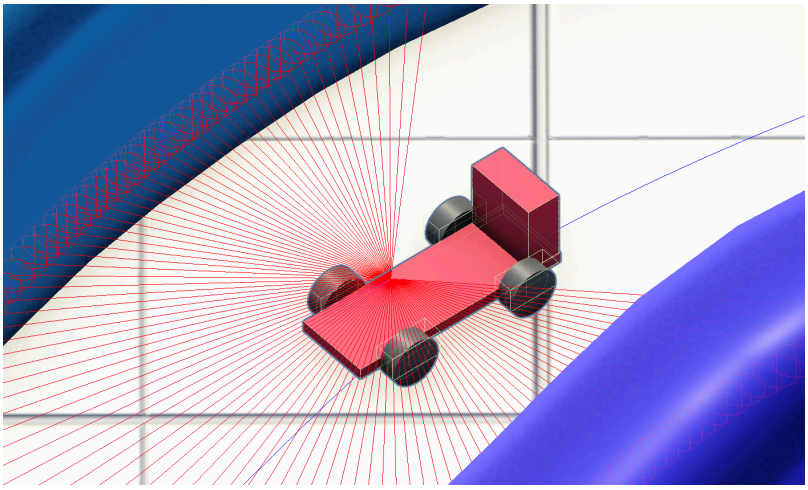
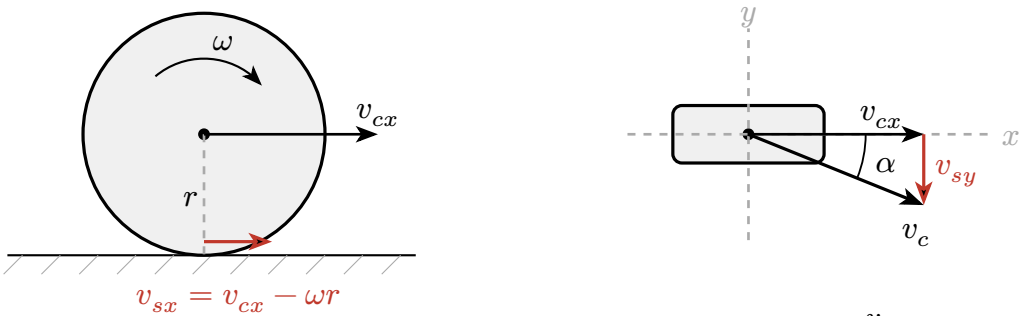


Figure 3.3: Custom Unity car's colliders: the body colliders handle collisions, and the four `WheelColliders` handle suspension and tyre forces.

3.1.2.2. The WheelCollider

In our custom simulator, each wheel of the RoboRacer is modelled as a `WheelCollider`, which simulates the wheel's suspension, rotation and tyre friction [47]. This kind of collider uses physics similar to that modelled by the semi-empirical "magic formula" wheel dynamics proposed by Pacejka et al. in [48], and further described in [49].

The tyre model is built on the idea of *slip*. *Longitudinal slip*, κ, is the ratio between the longitudinal slip velocity and the wheel center velocity. *Lateral slip*, $\tan(\alpha)$, is the ratio between the lateral slip velocity and the wheel center velocity. A wheel slips when it spins faster than the wheel center moves (wheelspin), turns slower than the wheel center moves (braking), or is dragged sideways (cornering). A slip value of 0 therefore indicates a freely rolling wheel, while a slip value of > 1 indicates that the point of contact has a higher velocity than the wheel center.



(a) Side view: longitudinal slip **(b)** Top view: lateral slip

Figure 3.4: Slip quantities. Longitudinal slip compares the slip velocity at the contact point with the wheel-centre velocity; lateral slip compares the sideways slip velocity with the forward velocity.

A tyre needs some slip to produce force. The `WheelCollider` computes the force from the slip using a friction curve (Figure 3.5). Force rises with slip until it reaches a peak, the *extremum*, which is the tyre's maximum grip. Beyond the peak, the tyre begins to slide and the force falls to a lower, constant value, the *asymptote*. Pushing the car harder therefore gives less force, not more, which is a regime a racing agent must learn to avoid. The curve is defined separately for the forward direction (accelerating and braking) and the sideways direction (cornering). Note that in real life, at low slip values, the rubber stretches instead of actually sliding on the road. This is modelled by the rising part of the curve at low slip values.

Figure 3.5 shows the difference between a Unity-style piecewise friction curve and a Pacejka "magic formula" curve that was fit using least-squares to approximate Unity's friction curve.

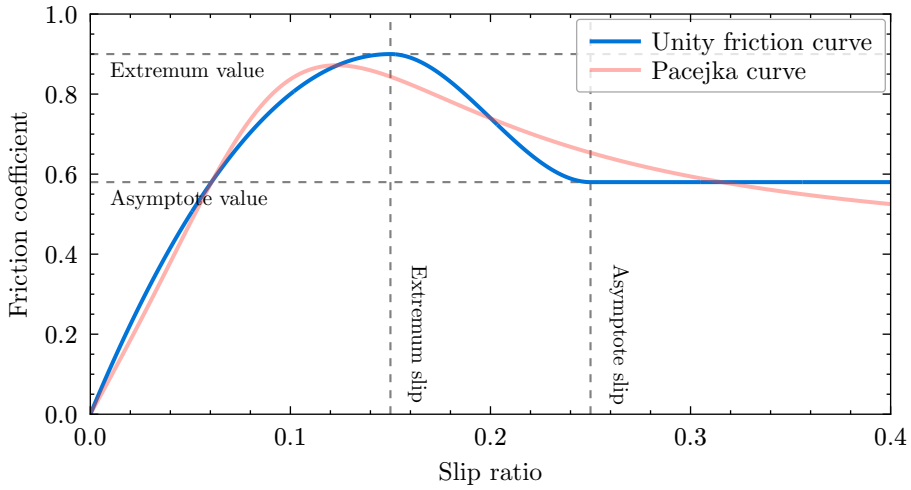


Figure 3.5: Approximated WheelCollider friction curves for the RoboRacer [47][6]

3.1.2.3. LiDAR

The vehicle is equipped with a simulated 2D LiDAR sensor, which is implemented using a `RayPerceptionSensor` from the ML-Agents toolkit. The sensor emits 101 rays in a fan pattern, covering a field of view of 270°. Each ray measures the distance to the nearest obstacle, such as walls or other vehicles, and returns a normalized value between 0 and 1, where 1 indicates no obstacle within the maximum range of the sensor (20m). The sensor also exposes a hit/miss flag for each ray, as well as flags for which object type was hit (opponent, wall, floor etc.).

3.1.3. Track generation

During training, a variety of tracks is used to prevent the policy from overfitting to any single layout. Training on diverse environments has been shown to improve generalisation to unseen environments [50], and is closely related to domain randomisation, where simulator parameters are randomised to produce more robust policies [51], [52]

3.1.3.1. Centerline generation

To this end, we implemented a procedural track generator in C# in Unity. The idea is a *mean-reverting random walk*: a polyline of N equal-length segments whose heading wanders randomly but is continually pulled back towards the heading of a *reference circle* of radius R. The result wanders freely but still forms a single closed loop of roughly predictable size.

The walk keeps track of two angles: the *reference heading* φ_i, the tangent direction of the reference circle, and a *heading offset* δ_i relative to it. The segment direction is $\theta_i =$

[6]The parameter values rendered in this chart are the default values configured in the AutoDrive simulator for the forward direction. See Appendix A for the values.

$\varphi_i + \delta_i$. Only the offset is random; the reference heading advances deterministically by $2\pi/N$ per step.

The walk starts at $p_0 = (R, 0)$ with $\delta_0 = 0$, i.e. tangent to the circle. Each segment has length $\ell = 2\pi R/N$, the circumference divided by the number of segments. For $i = 0, 1, ..., N-1$, with progress $t_i = (i+1)/N$:

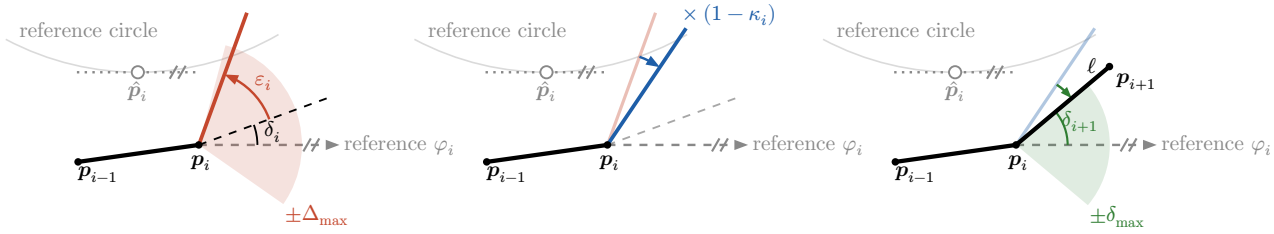
1. We calculate the reference heading, $\varphi_i = 2\pi t_i + \pi/2$, the tangent of the reference circle at $\hat{p}_i = R \cdot (\cos 2\pi t_i, \sin 2\pi t_i)$.
2. We take a random turn (Figure 3.6 a), drawing $\varepsilon_i \sim \mathcal{U}(-\Delta_{\max}, \Delta_{\max})$ and adding it to the offset.
3. As depicted in Figure 3.6 b, we shrink the offset the offset towards zero by a factor $1 - \kappa_i$, where

   $$\kappa_i = \kappa_0 + (1 - \kappa_0) t_i^s.$$

   The restoring strength κ_i starts at κ_0 and ramps up to exactly 1 on the final step. The exponent s controls how late the ramp kicks in.
4. In Figure 3.6 c, we limit the offset from the reference heading to prevent accumulation of large offsets, and advance by length l:

   $$\delta_{i+1} = \text{clip}((1 - \kappa_i)(\delta_i + \varepsilon_i), -\delta_{\max}, \delta_{\max}),$$

   $$p_{i+1} = p_i + \ell(\cos(\varphi_i + \delta_{i+1}), \sin(\varphi_i + \delta_{i+1})).$$



(a) Random turn (b) Mean reversion (c) Clip and step

Figure 3.6: One step of the track-generation walk. **(a)** A random turn $\varepsilon_i \sim \mathcal{U}(-\Delta_{\max}, \Delta_{\max})$ is added to the carried offset δ_i, measured from the reference heading φ_i. **(b)** A fraction κ_i of the accumulated offset is removed, pulling the heading towards the reference φ_i. **(c)** The offset is clipped to $\pm\delta_{\max}$, giving δ_{i+1}, and a step of length ℓ is taken to $\boldsymbol{p}_{i+1}$.

Early on, $\kappa_i \approx \kappa_0$ is small, so the offset behaves like a weakly damped random walk (an AR(1) process) and the track meanders. Near the end, $\kappa_i \to 1$ forces the heading back onto the reference tangent, so the walk heads towards the start point. Because the integrated heading still deviates slightly from the circle, p_N generally misses p_0 by a small gap. We close it by distributing the gap linearly along the track:

$$p_i \leftarrow p_i + \frac{i}{N}(p_0 - p_N), \quad i = 0, ..., N.$$

The gap is (usually) small, so this correction barely distorts the track.

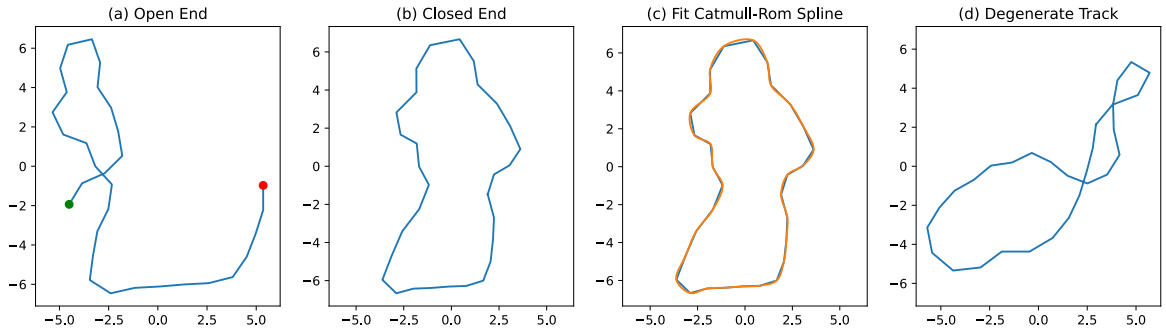


Figure 3.7: (a) Open end of the track before distributing the gap between p_0 (green) and p_N (red) **(b)** Closed track after distributing the gap **(c)** Catmull-Rom spline interpolation of the polyline **(d)** A degenerate track

This method is not immune to generating self-intersecting (degenerate) tracks, but the probability is low enough, that we simply discard any degenerate tracks and generate a new one. Empirically we find 50 tracks of 100 000 to be degenerate with our chosen parameters, so a 95%-confidence interval (C.I.) for $P(\text{degenerate track} \mid N = 30, \Delta_{\max} = 80°, \kappa_0 = 0.1, s = 3, \delta_{\max} = 90°)$ is $(0.14\%, 0.19\%)$ assuming a binomial distribution.

3.1.3.2. Track walls

Adding track walls to the generated centerline spline is a non-trivial problem. A naive approach is to scale the centerline's vertices outwards by a fixed multiple, or provide a fixed offset from the center, but this can create undrivably narrow sections, as visualised in Figure 3.8 (a) and (b) respectively. We therefore generate the walls using the Clipper2 library [53], which implements a polygon offsetting algorithm inspired by Vatti in [54]. The Clipper2 algorithm is robust to self-intersections, but may producce inset "islands" for large inset distances, or for tracks which are close to self-intersecting, as visualised in Figure 3.8 (c).

Empirically, we find that at 135 of the 100 000 tracks produce these islands (C.I. $(1.28\%, 1.42\%)$), so we simply discard any tracks with islands and re-generate them.

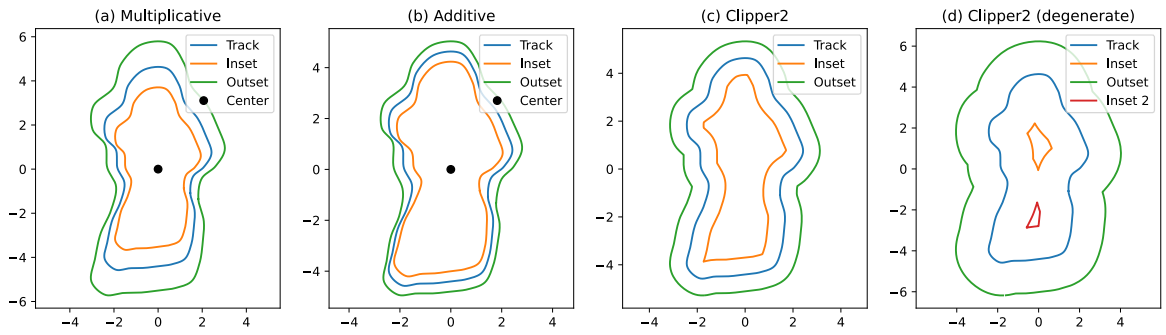


Figure 3.8: Track insets and outsets using different methods

3.2. Environment interface

At every control step (25 Hz, i.e. 40 ms, or four 100 Hz physics steps), each of the `num_areas` agents in an instance sends the following over gRPC:

- a 101-ray LiDAR scan, reduced to its normalised distance channel (the hit/miss and object-type flags are discarded by the environment wrapper, because the policy only uses distance), and
- its current steering angle and throttle.
- privledged quantities including
  - the agent's forward and lateral velocity (v_f and v_l) in m/s
  - the agent's yaw rate ω_y in rad/s
  - the track curvature[7] at 17 equally spaced points between 0.5m and 8m ahead of the agent along the centerline, in 1/m
  - the forward and lateral per-wheel slip values for each wheel, κ_x and $\tan(\alpha_x)$ with $x \in \{\mathrm{FR}, \mathrm{FL}, \mathrm{BR}, \mathrm{BL}\}$, which are unitless

The trainer stacks the 50 most recent scans for each agent (2 s of history at 25 Hz) and passes the stack, together with the steering and throttle, to the policy. The policy returns a steering and throttle command for each agent, which the simulator applies for the next control step. Privileged quantities (velocities, per-wheel slip, upcoming curvature) are sent alongside the observation, and are used only as auxiliary targets.

Because there are `num_inst` Unity instances, the trainer waits for all of them to report before running a single batched inference pass.

[7]The reciprocal of the radius of curvature is also known as the curvature. It is used to prevent large values on straight sections of the track

3.3. The Agent

The agent is a feedforward actor-critic network implemented in PyTorch, with the TorchRL library handling the action distribution and the PPO loss. It has no recurrent state, so the stack of past scans is its only source of temporal information. It consists of a shared *trunk* (Figure 3.9) followed by three independent heads (Figure 3.10), each of which takes as input the same 64-dimensional feature vector output from the trunk:

- the *policy head* outputs the parameters of the action distribution,
- the *value head* outputs the state-value estimate $\hat{V}^{\pi}(s)$, and
- the *auxiliary head* outputs predictions of the privileged targets

The trunk (Figure 3.9) has a convolutional part followed by a dense part. The input is a tensor of shape 50×101 (time steps by rays). Four 1D convolutions with ReLU activations slide along the ray axis and treat the time steps as input channels, so the first layer mixes the whole history at each angular position. They have 32, 16, and 16 output channels respectively; all have the same kernel size of 8 (roughly 21° of FOV). The output is flattened and concatenated with the current steering angle and throttle, and passed through fully connected layers of 128, 72 and 64 units with ReLU activations, followed by layer normalisation of the 64-dimensional output to prevent gradient explosion when the car is under heavy acceleration, pointing the LiDAR above the track walls and resulting in large ray lengths as input.

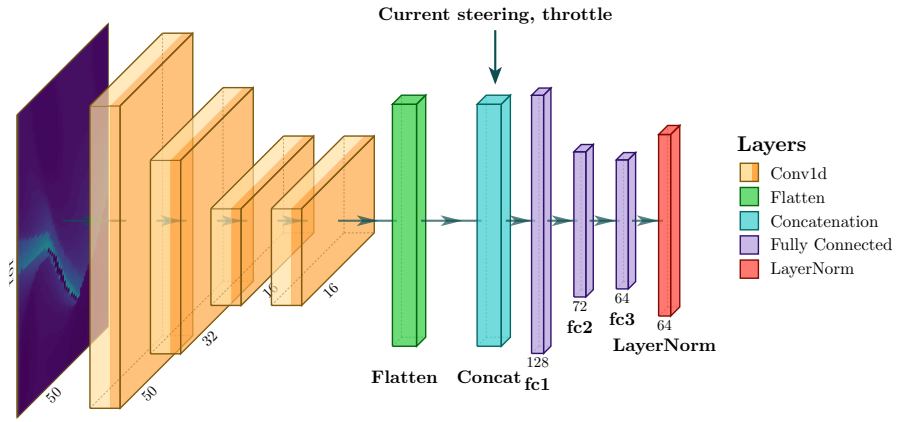
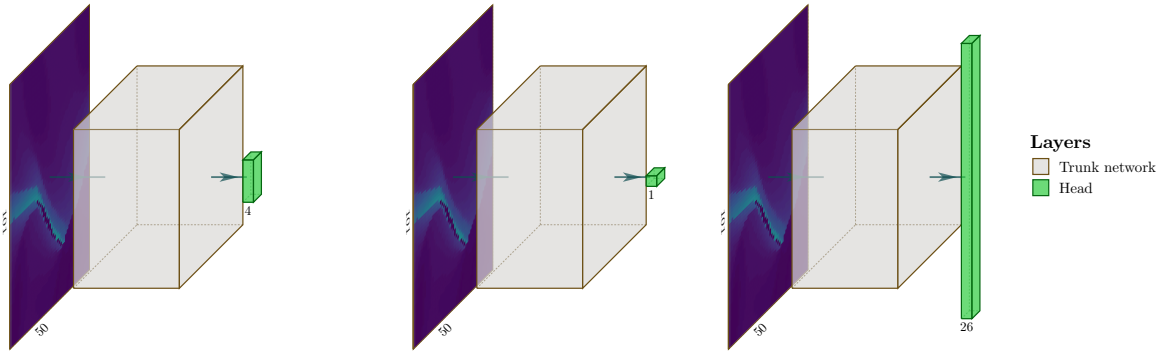


Figure 3.9: Trunk trunk network. A 50-frame LiDAR stack passes through four 1D convolutions; the flattened output is concatenated with the current steering and throttle and passed through three dense layers and a layer norm to give a 64-dimensional feature vector.



(a) Policy head **(b)** Value head **(c)** Auxiliary head

Figure 3.10: Heads attached to the shared 64-dimensional trunk output. Each head is a single linear layer (no activation): the policy head outputs 4 values (location and scale for throttle and steering), the value head 1, and the auxiliary head 27 (one per privileged target).

3.3.1. Action Distributions

The policy head outputs a location μ and a raw scale value x for each of throttle and steering. Each action is sampled independently from a tanh-squashed Gaussian,

$$a = \tanh(u), \quad u \sim \mathcal{N}(\mu, \sigma^2),$$

so that $a \in (-1, 1)$. To keep the location from drifting arbitrarily far, it is additionally soft-clamped to roughly $[-5, 5]$ (`tanh_loc=True` in TorchRL's `TanhNormal`).

The standard deviation is made positive with a softplus, with a small floor: $\sigma = \text{softplus}(x) + 10^{-3}$.

The policy head's linear layer uses a weight initialisation with gain 0.01 and zero bias, so every output is close to 0 at the start. Hence $\mu \approx 0$, and both actions are centred near the middle of their range ($\tanh(0) = 0$).

This is important because the derivative of the squashing function is

$$\frac{\mathrm{d}a}{\mathrm{d}u} = 1 - \tanh^2(u),$$

which goes to 0 as $|u|$ grows. If the head's outputs start large, samples land in the saturated region, the gradient of the policy loss with respect to μ is almost zero, and the policy cannot recover. In early experiments this is exactly what happened: the car locked at full throttle and full steering in either the positive or negative direction.

3.4. Auxiliary Targets

Each frame, the environment emits a set of privledged quantities to the trainer, which are used as auxiliary targets to improve the policy's internal representation of the environment. These targets are listed in Table 3.2.

Table 3.2: The auxiliary targets to be predicted by the agent

Target	Units	Description
Forward velocity v_f	m/s	The component of the agent's velocity in the direction of its forward axis
Lateral velocity v_l	m/s	The component of the agent's velocity in the direction of its lateral axis
Yaw rate ω_y	rad/s	The rate of change of the agent's heading about its vertical axis
Track curvature ahead c_d	1/m	The curvature of the track centerline at d meters ahead of the agent, measured as the reciprocal of the radius of curvature
Per-wheel longitudinal slip κ_x	unitless	The longitudinal slip of wheel $x \in \{\mathrm{FR}, \mathrm{FL}, \mathrm{BR}, \mathrm{BL}\}$
Per-wheel lateral slip $\tan(\alpha_x)$	unitless	The lateral slip of wheel $x \in \{\mathrm{FR}, \mathrm{FL}, \mathrm{BR}, \mathrm{BL}\}$

In order to ensure the targets are on a similar scale, the targets are normalised to $\sigma = 1, \mu = 0$ using a rollout generated from a pure PPO policy.

Because the auxiliary targets are heavy-tailed, we use the Huber loss to reduce the influence of outliers and prevent the policy from (a) overfitting to them; and (b) producing large gradients that destabilise training.

3.5. Training

The agent is trained using the PPO algorhtm (Section 2.3.2) using Generalised Advantage Estimation. The total loss minimised is

$$\mathcal{L} = \mathcal{L}_{\mathrm{PPO}} + c_v \mathcal{L}_V - c_e H[\pi_\theta] + \sum_{k=1}^{K} w^{(k)} \mathcal{L}_{\mathrm{aux}}^{(k)},$$

where $\mathcal{L}_V$ is the value loss, H the policy entropy and $\mathcal{L}_{\mathrm{aux}}^{(k)}$ the loss of the k-th auxiliary target (Huber loss on normalised targets). The baseline model uses the same loss with all $w^{(k)} = 0$, so the two differ only in the auxiliary term. Each iteration collects a 256-step rollout from all agents, then runs 4 epochs of minibatch updates with a learning rate of 1×10^{-4}.

Once the car collides with a wall, the episode is terminated and the agent is reset to a random position along the centerline with a heading offset sampled from $U(-20°, 20°)$ with a newly generated track every 3 resets. The track direction is also randomised to prevent the agent from forming a bias towards a particular direction. Every 45s of sim time, the episode is truncated and the agent is again reset. Because the tracks have a radius of roughly 6m, and the car travels at around 3m/s, the agent is afforded enough time to complete approximately 4 laps before the episode is truncated. In all examples, we train the agent fo 600 timesteps, where it experiences $\frac{256 \text{ control steps}}{25 \text{ Hz}} \cdot 48 \text{ agents} =$ 8.2 mins of driving time.

3.6. Metrics

3.6.1. Progress

Let L be the total length of the track centerline, and let $\xi_t \in [0, L)$ be the agent's *position along the track* at time t: the arc length, in metres, from the start of the centerline spline to the point on the spline closest to the agent. The *progress per step* is the distance travelled along the centerline between consecutive timesteps, i.e. the change in this position, $\xi_t - \xi_{t-1}$.

Because the track is a closed loop, ξ wraps from L to 0 at the finish line, which would cause the raw difference $\xi_t - \xi_{t-1}$ to jump to $-L$. We therefore define the progress per step as $\text{wrap}(\xi_t - \xi_{t-1})$, where

$$\text{wrap}(\Delta) = \left(\left(\Delta + \frac{L}{2} \right) \bmod L \right) - \frac{L}{2}$$

maps the difference back into the interval $\left[-\frac{L}{2}, \frac{L}{2}\right)$. Crossing the finish line thus yields a continuous positive progress value rather than a large negative spike, and moving backward along the track yields negative progress.

3.6.2. Crash rates

The *crash rate per episode* is the fraction of episodes that end in a collision, instead of reaching the maximum episode length. The *crash rate per timestep* is the fraction of timesteps in which a collision occurs.

3.6.3. Reward function

The agent receives a dense reward at each timestep that encourages fast, collision-free progress along the track:

$$r_t = w_p \cdot \text{wrap}(\xi_t - \xi_{t-1}) - w_c \cdot c_t$$

The first term rewards forward progress, using the progress per step defined above. The second term penalizes collisions: the indicator variable c_t equals 1 if a collision occurs with an obstacle at time t, and 0 otherwise.

The weights w_p (reward per metre of progress) and w_c (penalty per collision) balance speed against safety based on the risk tolerance of the application. We set $w_p = 1.0$ and $w_c = 1.0$. Because an episode terminates immediately upon collision, there is already a strong implicit penalty for crashing, allowing w_c to remain relatively low.

3.6.4. Area Under the Curve (AUC)

The *Area Under the Curve* (AUC) evaluates sample efficiency and learning stability by calculating the definite integral of a performance metric $y(t)$ (e.g., episodic return or progress per step) over the total training time or timesteps T:

$$\text{AUC} = \int_0^T y(t)\,\mathrm{d}t$$

In practice, evaluation metrics are recorded at discrete interval checkpoints $t_0, t_1, ..., t_N$, and the AUC is computed using the trapezoidal rule:

$$\text{AUC} \approx \sum_{i=1}^{N} \frac{y(t_{i-1}) + y(t_i)}{2}(t_i - t_{i-1})$$

A higher AUC indicates faster learning and superior performance throughout the entire training process rather than merely achieving high final performance.

3.7. Trainer Performance Tuning

Training was performed on a single machine with an AMD Ryzen Threadripper PRO 3995WX CPU (64 cores, 128 hardware threads) and 128 GB of RAM. To ensure experiments finish in a practical time, we first measured how best to use this hardware, aiming to maximise *aggregate* throughput: environment timesteps collected and learned from per second, summed across all models training simultaneously.

We ran a grid search over the parameters in Table 3.3: concurrent trainers (2, 4, 8, 12), Python threads per trainer (8, 16) and parallel environments per trainer (4, 8, 10, 16). For each configuration, `concurrent_trainers` identical processes were launched at once, each running five PPO iterations (a 256-frame rollout plus four epochs of mini-batch updates each iteration) with checkpointing and TensorBoard logging disabled.

Configurations with more than 8 trainers and 16 environments (>128 concurrent Unity instances) exhausted memory and were excluded, leaving 28 single-run configurations. Throughput per trainer was computed from the mean wall-clock time per iteration:

$$\text{throughput} = \frac{\text{rollout length} \times \text{parallel envs} \times \text{areas per env}}{\text{seconds per iteration}}$$

We report the throughput per trainer, showing how much each model slows as the machine gets busier, and summed across trainers, showing where adding trainers stops paying off.

As shown in Appendix C, total throughput peaks at 8 trainers (4172 timesteps/s with 8 environments and 16 threads) and falls at 12 (at most 3853 timesteps/s), while per-trainer throughput roughly halves from 2 to 8 trainers. The best settings use $8 \times 16 = 128$ python threads in total, matching the hardware thread count; at 12 trainers the 16-thread configurations are oversubscribed and fall behind the 8-thread ones. We therefore chose (`concurrent_trainers`, `parallel_envs`, `python_threads`) = (8, 8, 16), though its margin over the next-best configurations (4020 and 3962 timesteps/s) is within likely run-to-run variation. Using `num_areas` $\in \{8, 12\}$ per environment in this configuration did not help (3844 and 3999 timesteps/s).

By default, Unity spawns `n_cpus` garbage-collector threads; we set this to 8 with the `-gc-helper-count` launch argument for all runs.

Table 3.3: Parameters varied in the training performance grid search.

Parameter	Values considered	Description
`concurrent_trainers`	$\{2, 4, 8, 12\}$	Number of independent models training simultaneously, each in its own process.
`python_threads`	$\{8, 16\}$	Threads available to each trainer's PyTorch, OpenMP, and OpenBLAS thread pools.
`parallel_envs`	$\{4, 8, 10, 16\}$	Number of Unity environment instances per trainer. Each contains 6 training areas, so each instance steps 6 agents in parallel.

Bibliography

[1] J. Hankey *et al.*, "Identification and evaluation of driver errors: Task c report, driver error taxonomy development," *Project no. Dtfh-61-97-c-00051. Virginia Tech Transportation Institute, Blacksburg, VA*, 1999.

[2] A. L. Medina, S. E. Lee, W. W. Wierwille, and R. J. Hanowski, "Relationship between Infrastructure, Driver Error, and Critical Incidents," *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 48, no. 16, pp. 2075–2079, 2004, doi: 10.1177/154193120404801661.

[3] B. Olofsson and L. Nielsen, "Using Crash Databases to Predict Effectiveness of New Autonomous Vehicle Maneuvers for Lane-Departure Injury Reduction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 6, pp. 3479–3490, 2021, doi: 10.1109/TITS.2020.2983553.

[4] C. J. G. V. Driel and B. V. Arem, "The Impact of a Congestion Assistant on Traffic Flow Efficiency and Safety in Congested Traffic Caused by a Lane Drop," *Journal of Intelligent Transportation Systems*, vol. 14, no. 4, pp. 197–208, 2010, doi: 10.1080/15472450.2010.516226.

[5] H. Igliński and M. Babiak, "Analysis of the Potential of Autonomous Vehicles in Reducing the Emissions of Greenhouse Gases in Road Transport," *Procedia Engineering*, vol. 192, pp. 353–358, 2017, doi: https://doi.org/10.1016/j.proeng.2017.06.061.

[6] J. Chen, S. E. Li, and M. Tomizuka, "Interpretable End-to-end Urban Autonomous Driving with Latent Deep Reinforcement Learning." [Online]. Available: https://arxiv.org/abs/2001.08726

[7] B. Rafiee, J. Jin, J. Luo, and A. White, "What makes useful auxiliary tasks in reinforcement learning: investigating the effect of the target policy." [Online]. Available: https://arxiv.org/abs/2204.00565

[8] A. Y. Ng, D. Harada, and S. J. Russell, "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping," in *Proceedings of the Sixteenth International Conference on Machine Learning*, in ICML '99. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1999, pp. 278–287.

[9] A. Kendall, Y. Gal, and R. Cipolla, "Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, Jun. 2018, pp. 7482–7491. [Online]. Available: https://arxiv.org/abs/1705.07115

[10] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998.

[11] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[12] R. Bellman, "A Markovian Decision Process," *Journal of Mathematics and Mechanics*, vol. 6, no. 5, pp. 679–684, 1957.

[13] T. P. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," Sep. 2015. [Online]. Available: https://ar5iv.arxiv.org/html/1509.02971

[14] S. P. Singh, T. Jaakkola, and M. I. Jordan, "Learning Without State-Estimation in Partially Observable Markovian Decision Processes," in *Machine Learning Proceedings 1994*, W. W. Cohen and H. Hirsh, Eds., San Francisco (CA): Morgan Kaufmann, 1994, pp. 284–292. doi: https://doi.org/10.1016/B978-1-55860-335-6.50042-8.

[15] P. Thomas, "Bias in Natural Actor-Critic Algorithms," in *Proceedings of the 31st International Conference on Machine Learning*, E. P. Xing and T. Jebara, Eds., in Proceedings of Machine Learning Research, vol. 32. Beijing, China: PMLR, 2014, pp. 441–448. [Online]. Available: https://proceedings.mlr.press/v32/thomas14.html

[16] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3, pp. 229–256, May 1992, doi: 10.1007/BF00992696.

[17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms." [Online]. Available: https://arxiv.org/abs/1707.06347

[18] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation." [Online]. Available: https://arxiv.org/abs/1506.02438

[19] M. Jaderberg *et al.*, "Reinforcement Learning with Unsupervised Auxiliary Tasks," in *International Conference on Learning Representations (ICLR 2017)*, 2017. doi: 10.48550/arXiv.1611.05397.

[20] E. Hüllermeier, T. Fober, and M. Mernberger, "Inductive Bias," in *Encyclopedia of Systems Biology*, W. Dubitzky, O. Wolkenhauer, K.-H. Cho, and H. Yokota, Eds., New York, NY: Springer, 2013, p. 1018. doi: 10.1007/978-1-4419-9863-7_927.

[21] R. Caruana, "Multitask Learning," *Machine Learning*, vol. 28, pp. 41–75, Jul. 1997, doi: 10.1023/A:1007379606734.

[22] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven exploration by self-supervised prediction," in *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, in ICML'17. Sydney, NSW, Australia: JMLR.org, 2017, pp. 2778–2787.

[23] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," *arXiv preprint*, Jul. 2019, doi: 10.48550/arXiv.1907.10902.

[24] J. Bergstra and Y. Bengio, "Random Search for Hyper-Parameter Optimization," *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012, [Online]. Available: https://jmlr.org/papers/v13/bergstra12a.html

[25] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian Optimization of Machine Learning Algorithms," in *Advances in Neural Information Processing Systems*, 2012, pp. 2951–2959. [Online]. Available: https://arxiv.org/abs/1206.2944

[26] J. S. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for Hyper-Parameter Optimization," in *Advances in Neural Information Processing Systems*, 2011. [Online]. Available: https://papers.nips.cc/paper_files/paper/2011/hash/86e8f7ab32cfd12577bc2619bc635690-Abstract.html

[27] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization," *Journal of Machine Learning Research*, vol. 18, pp. 1–52, 2018, [Online]. Available: https://arxiv.org/abs/1603.06560

[28] L. Li *et al.*, "A System for Massively Parallel Hyperparameter Tuning," in *Proceedings of Machine Learning and Systems*, 2020, pp. 230–246. [Online]. Available: https://arxiv.org/abs/1810.05934

[29] A. Kendall and Y. Gal, "What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?," *arXiv preprint*, Mar. 2017, [Online]. Available: https://arxiv.org/abs/1703.04977

[30] L. Kirchdorfer, T. Sesterhenn, C. Bartelt, H. Stuckenschmidt, L. Schott, and J. M. Köhler, "Investigating Uncertainty Weighting for Multi-Task Learning: Insights and Analytical Alternative," *International Journal of Computer Vision*, vol. 134, no. 8, Dec. 2025, doi: 10.1007/s11263-025-02625-x.

[31] P. J. Huber, "Robust Estimation of a Location Parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, Mar. 1964, doi: 10.1214/aoms/1177703732.

[32] G. Alain and Y. Bengio, "Understanding intermediate layers using linear classifier probes." [Online]. Available: https://arxiv.org/abs/1610.01644

[33] A. Anand, E. Racah, S. Ozair, Y. Bengio, M.-A. Côté, and R. D. Hjelm, "Unsupervised State Representation Learning in Atari." [Online]. Available: https://arxiv.org/abs/1906.08226

[34] Y. Belinkov, "Probing Classifiers: Promises, Shortcomings, and Advances." [Online]. Available: https://arxiv.org/abs/2102.12452

[35] T. McGrath *et al.*, "Acquisition of chess knowledge in AlphaZero," *Proceedings of the National Academy of Sciences*, vol. 119, no. 47, p. e2206625119, 2022, doi: 10.1073/pnas.2206625119.

[36] W. Gurnee and M. Tegmark, "Language Models Represent Space and Time." [Online]. Available: https://arxiv.org/abs/2310.02207

[37] G. Lample and D. S. Chaplot, "Playing FPS Games with Deep Reinforcement Learning," in *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, AAAI Press, 2017, pp. 2140–2146. [Online]. Available: https://arxiv.org/abs/1609.05521

[38] A. Dosovitskiy and V. Koltun, "Learning to act by predicting the future," *arXiv preprint arXiv:1611.01779*, 2016.

[39] P. Mirowski *et al.*, "Learning to Navigate in Complex Environments." [Online]. Available: https://arxiv.org/abs/1611.03673

[40] E. Shelhamer, P. Mahmoudieh, M. Argus, and T. Darrell, "Loss is its own Reward: Self-Supervision for Reinforcement Learning." [Online]. Available: https://arxiv.org/abs/1612.07307

[41] Unity Technologies, *Unity ML-Agents Toolkit*. (2017). GitHub. Accessed: Oct. 13, 2026. [Online]. Available: https://github.com/Unity-Technologies/ml-agents

[42] A. Juliani *et al.*, "Unity: A General Platform for Intelligent Agents," 2018. [Online]. Available: https://arxiv.org/abs/1809.02627

[43] B. D. Evans *et al.*, "Unifying F1TENTH Autonomous Racing: Survey, Methods and Benchmarks." [Online]. Available: https://arxiv.org/abs/2402.18558

[44] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1TENTH: An Open-source Evaluation Environment for Continuous Control and Reinforcement Learning," in *NeurIPS 2019 Competition and Demonstration Track*, 2020, pp. 77–89.

[45] T. V. Samak, C. V. Samak, and M. Xie, "AutoDRIVE Simulator: A Simulator for Scaled Autonomous Vehicle Research and Education," in *2021 2nd International Conference on Control, Robotics and Intelligent System*, in CCRIS'21. Qingdao, China: Association for Computing Machinery, 2021, pp. 1–5. doi: 10.1145/3483845.3483846.

[46] "Customised Usage Example." Accessed: Oct. 14, 2026. [Online]. Available: https://f1tenth-gym.readthedocs.io/en/v1.0.0/customized_usage.html#random-track-generator-beta

[47] "Unity Documentation." [Online]. Available: https://docs.unity3d.com/6000.5/Documentation/ScriptReference/WheelFrictionCurve.html

[48] H. B. Pacejka and E. Bakker, "THE MAGIC FORMULA TYRE MODEL," *Vehicle System Dynamics*, vol. 21, no. sup001, pp. 1–18, 1992, doi: 10.1080/00423119208969994.

[49] H. Pacejka, *Tyre and Vehicle Dynamics*. in Automotive Engineering Series. Elsevier Science & Technology Books, 2002. [Online]. Available: https://books.google.co.za/books?id=pQNUAAAAMAAJ

[50] K. Cobbe, O. Klimov, C. Hesse, T. Kim, and J. Schulman, "Quantifying Generalization in Reinforcement Learning," in *International Conference on Machine Learning (ICML)*, 2019, pp. 1282–1289.

[51] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.

[52] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[53] A. Johnson, *Clipper2: Polygon Clipping, Offsetting & Triangulation in C++, C# and Delphi*. (2022). GitHub. Accessed: Oct. 08, 2026. [Online]. Available: https://github.com/AngusJohnson/Clipper2

[54] B. R. Vatti, "A generic solution to polygon clipping," *Communications of the ACM*, vol. 35, no. 7, pp. 56–63, Jul. 1992, [Online]. Available: https://dl.acm.org/doi/10.1145/129902.129906

Appendix

Appendix A WheelCollider properties

Table 6.1: Relevant parameters of the WheelCollider model. * = forward & sideways

Property	Description	Value for RoboRacer
`radius`	Radius of the wheel	5.8 cm
`width`	Radius of the wheel	5.8 cm
`mass`	Mass of each wheel; determines its rotational inertia	109 g
`suspension_distance`	Maximum travel of the suspension along the wheel's local Y axis	5 cm
`suspension_spring`	Spring stiffness pulling the wheel toward its target position	500 N/m
`suspension_damper`	Damping that resists the speed of suspension movement	100 (N · s/m)
`extremum_slip`*	Slip at which the friction curve peaks (maximum grip)	0.15 fwd / 0.01 side
`extremum_value`*	Friction at the peak of the curve	0.9 fwd / 1 side
`asymptote_slip`*	Slip beyond which the tyre is fully sliding	0.25 fwd / 0.1 side
`asymptote_value`*	Friction while sliding	0.58 fwd / 0.5 side
`stiffness`*	Multiplier applied to the whole friction curve	0.8 fwd / 1 side

Appendix B Auxiliary Head Regression Results

Appendix C Training Performance Benchmark Results

Table 6.2: Training performance results, sorted by throughput.

Concurrent trainers	Python threads	Parallel envs[8]	Throughput (timesteps/sec)	Speedup vs realtime[9]
8	16	8	4172.37	166.89x
8	16	4	4020.17	160.81x
8	16	8	3999.46	159.98x
8	16	10	3962.12	158.48x
12	8	4	3853.04	154.12x
8	16	8	3843.59	153.74x
12	16	10	3805.19	152.21x
8	8	4	3802.34	152.09x
12	8	8	3751.41	150.06x
12	8	10	3717.67	148.71x
12	16	8	3582.39	143.3x
4	16	10	3428.19	137.13x
12	16	4	3302.99	132.12x
8	8	8	3207.98	128.32x
4	16	8	3201.63	128.07x
8	8	10	3164.72	126.59x
4	16	4	2684.4	107.38x
4	8	4	2414.64	96.59x
2	16	10	2206.84	88.27x
4	8	16	2086.99	83.48x
4	16	16	2075.62	83.02x
2	16	8	2034.77	81.39x
4	8	10	1884.89	75.4x
4	8	8	1830.29	73.21x
2	16	4	1557.77	62.31x
2	8	4	1385.09	55.4x
2	8	16	1122.47	44.9x
2	16	16	1122.1	44.88x
2	8	10	1013.7	40.55x
2	8	8	977.93	39.12x

[8]It was not possible to run 16 parallel envs on 12 trainers, because the machine ran out of memory.
[9]Assuming 25 Hz decision frequency